Quality Log

DEF-001 — Missing Image Upload on Authentication Pages

Fix Steps & Approach 1. Added image assets folder to `frontend/src/assets` 2. Created `LoginComponent` with background image support 3. Configured `RegisterComponent` with logo placeholder 4. Implemented responsive image sizing using CSS media queries 5. Added fallback placeholder for missing images

Testing Methodology Manual UI testing across browsers (Chrome, Firefox, Safari, Edge). Cross-browser screenshot comparison. Mobile responsiveness testing on various screen sizes.

Verification Criteria - Image displays correctly on login page - Sign up page shows proper branding logo - Images responsive on mobile devices - No broken image links in console

QA Sign-off: Passed

Regression Testing Tested on Android and iOS browsers. No CSS conflicts with other components. Page load time unaffected.

Notes Asset compression applied. Images optimized to <100 KB each. CDN ready.

DEF-002 — Car Listing Confirmation Missing

Fix Steps & Approach 1. Modified `CarController.addCar()` to return `ResponseEntity` with success message 2. Created `CarResponse` DTO with message field 3. Added notification logic to display 'pending approval' status 4. Implemented toast notification in frontend `CarListingComponent` 5. Updated validation to trigger message only on successful submission

Testing Methodology Unit testing for `CarService.saveCar()`. Integration testing with database transaction. Frontend component testing with Jasmine.

Verification Criteria - Success message displays immediately after listing submission - Message contains 'awaiting admin approval' text - No message shown if validation fails - Toast notification properly timed

QA Sign-off: Passed

Regression Testing Verified with invalid inputs (negative price, missing fuel type). No false positives.

Notes Tested with concurrent submissions. No race condition issues.

DEF-003 — Seller Review Feature Not Implemented

Fix Steps & Approach 1. Created `ReviewController` with POST endpoint for seller reviews 2. Implemented `SellerReview` entity with JPA annotations 3. Added `ReviewRepository` extending `JpaRepository` 4. Created `ReviewService` with rating validation (1–5 range) 5. Implemented `ReviewRepository.findBySeller()` and `findByReviewerAndSeller()` for upsert logic 6. Added frontend `ReviewDialog` component with star rating UI

Testing Methodology Unit tests for `ReviewService` validation. Integration tests for `ReviewController` endpoints. Frontend component testing with mock data. E2E testing with actual buyer-seller flow.

Verification Criteria - Review submission returns HTTP 201 - Rating stored in database within range 1–5 - Duplicate review updates existing record - Average rating calculation accurate - Ineligible users rejected with HTTP 403

QA Sign-off: Passed

Regression Testing Tested review with <1 and >5 ratings (rejected). Tested self-review (rejected). Tested without approved purchase (rejected).

Notes Average rating calculated correctly across 10+ reviews. No database constraint violations.

DEF-004 — Receipt Generator Missing for Admin

Fix Steps & Approach 1. Created `ReceiptGenerator` utility class for PDF generation using iText library 2. Implemented `/api/admin/orders/{orderId}/receipt` endpoint in `AdminController` 3. Added `ReceiptResponse` DTO with all order and payment details 4. Created `ReceiptService` with formatting logic for payment and order info 5. Integrated file download stream in HTTP response with proper headers

Testing Methodology Unit testing for `ReceiptGenerator` formatting. Integration testing for endpoint response format. Manual admin testing downloading multiple receipts.

Verification Criteria - Receipt PDF generated successfully - Download triggers browser download dialog - Receipt contains all required fields (buyer, seller, car, payment details) - PDF readable and properly formatted - Admin can download completed and pending orders

QA Sign-off: Passed

Regression Testing Downloaded 15+ receipts. File naming consistent. No PDF corruption.

Notes Tested with special characters in names. Large order IDs handled correctly.

DEF-005 — Admin Dashboard Analytics Graphs Not Rendering

Fix Steps & Approach 1. Fixed `DashboardService` graph data structure to include all 30 days of data 2. Corrected date formatting to ISO 8601 standard in response 3. Updated frontend Chart.js configuration with proper data binding 4. Implemented null-safe data processing for missing days 5. Added chart legend and tooltip formatting 6. Optimized date-based sorting in `ordersPerDay` and `revenuePerDay` arrays

Testing Methodology Unit testing for date range calculation. Integration testing for database query performance. Frontend component testing with mock data and real API responses. Visual regression testing.

Verification Criteria - All 5 chart types render (orders, revenue, cars, order-status, car-approval) - Date labels display correctly - No data points missing for 30-day period - Tooltip shows accurate values - Charts responsive on different screen sizes

QA Sign-off: Passed

Regression Testing Tested with 0-data days (still renders). Tested with 1000+ orders (performance acceptable). Mobile rendering verified.

Notes Chart performance: <500 ms render time. No memory leaks on repeated rendering.

DEF-006 — Admin Cannot Modify User Profile Details

Fix Steps & Approach 1. Created PATCH endpoint in `AdminController` for user updates 2. Implemented `AdminService.updateUser()` with field-level validation 3. Added security check to prevent admin from modifying own role 4. Implemented email and phone uniqueness validation against database 5. Created password reset logic with new hashing when `newPassword` provided 6. Invalidated sessions on password change to force re-login

Testing Methodology Unit testing for each field update independently. Integration testing for constraint violations (duplicate email, phone). Authorization testing for non-admin access.

Verification Criteria - Admin successfully updates user name - Email change rejected if duplicate - Phone change rejected if duplicate - Password change invalidates active sessions - Non-admin receives 403 error - Updated fields persist in database

QA Sign-off: Passed

Regression Testing Tested updating all fields simultaneously and individually. Tested with same values (no unnecessary DB writes). Tested concurrent updates.

Notes No SQL injection vulnerabilities. No session leakage after password reset.

DEF-007 — Insufficient Password Validation Rules

Fix Steps & Approach 1. Added `PasswordValidator` annotation with minimum 10-character constraint 2. Implemented validation in `RegisterRequest` and `UserUpdateRequest` 3. Added regex pattern requiring mix of alphanumeric and special chars 4. Created custom exception `PasswordValidationException` 5. Updated error message to clearly state password requirements 6. Added frontend validation to show real-time password strength indicator

Testing Methodology Unit testing for each validation rule. Boundary testing (9 chars, 10 chars, 11 chars). Integration testing for both registration and admin password reset.

Verification Criteria - Passwords <10 chars rejected with 400 error - 10+ char passwords accepted - Special char requirement enforced - Error message clearly describes requirement - Frontend shows strength meter - Same password cannot be reused within 3 months

QA Sign-off: Passed

Regression Testing Tested with 9, 10, and 50 character passwords. Tested with only letters, only numbers, special chars only.

Notes Bcrypt hashing strength verified. No plaintext passwords in logs.

DEF-008 — Wishlist Persistence Issue

Fix Steps & Approach 1. Configured Spring Session with Redis for distributed session management 2. Implemented `WishlistItem` entity with proper JPA cascade settings 3. Fixed `WishlistRepository` query methods for proper list retrieval 4. Added transactional boundaries to wishlist add/remove operations 5. Implemented session listener to persist wishlist on logout 6. Added database fallback when Redis cache unavailable

Testing Methodology Unit testing for `WishlistService` add and remove. Integration testing with database and Redis. Session persistence testing across browser refreshes.

Verification Criteria - Wishlist items persist after logout - Items retrieve correctly after page refresh - Adding duplicate item is idempotent - Remove operation atomic and immediate - Works with Redis unavailable (falls back to DB)

QA Sign-off: Passed

Regression Testing Tested clearing cookies and returning. Tested killing browser session. Tested server restart with items still intact.

Notes Data integrity verified: no orphaned wishlist items. Cache consistency checked.

DEF-009 — Payment Gateway Integration Incomplete

Fix Steps & Approach 1. Enhanced `SimulatedPaymentGateway` to properly mock payment responses 2. Updated `PurchaseService` to handle success and failure scenarios 3. Implemented `PaymentRepository` to persist payment records with status 4. Added transaction rollback on payment failure to release car availability 5. Created `OrderService` logic to update order status based on payment result 6. Implemented notification service to alert buyer of payment outcome

Testing Methodology Unit testing for success and failure payment flows. Integration testing for order creation and payment status updates. E2E testing for full purchase workflow.

Verification Criteria - Successful payment creates `PENDING_ADMIN_APPROVAL` order - Failed payment rolls back transaction and returns error - Payment status correctly stored in database - Order status reflects payment state - Buyer receives appropriate notification

QA Sign-off: Passed

Regression Testing Tested with `fail_` prefix tokens for failure case. Tested with valid tokens for success. Tested concurrent payments.

Notes Payment records immutable after creation. No duplicate payment processing.

DEF-010 — Recent Views Feature Not Tracking

Fix Steps & Approach 1. Created `RecentView` entity with user, car, and timestamp fields 2. Implemented `RecentViewRepository` with `findLatestByUser()` query 3. Added `RecentViewService` to record view on each car detail access 4. Configured cascading delete when user is removed 5. Added endpoint `GET /api/cars/recent` returning last 10 views 6. Implemented frontend service to populate 'Recent Views' section

Testing Methodology Unit testing for `RecentView` entity persistence. Integration testing for concurrent view tracking. Frontend component testing for display logic.

Verification Criteria - View recorded immediately after accessing car detail page - Last 10 views appear in correct order - View count increments on car entity - Old views removed after 10 new views - Works across multiple sessions

QA Sign-off: Passed

Regression Testing Tested with rapid consecutive views of same car (no duplicates). Tested view tracking disabled for admin.

Notes Memory usage optimized: only last 10 views cached.

DEF-011 — Car Comparison Missing Condition Fields

Fix Steps & Approach 1. Updated `CarResponse` DTO to include `condition`, `fuelType`, `transmission`, `bodyType`, `engineCc`, `numberOfOwners`, `insured`, `pucValid` 2. Modified `/api/cars/compare` endpoint to return complete Car objects 3. Updated database queries to eagerly load all condition fields 4. Created frontend comparison table component with all fields 5. Added field formatting for technical specs (e.g., `engineCc` displayed as '1497 cc') 6. Implemented side-by-side styling with colour-coding for differences

Testing Methodology Unit testing for `CarResponse` field population. Integration testing for comparison endpoint. Frontend component testing with mock car data.

Verification Criteria - Comparison returns both cars with all fields populated - All condition fields visible in side-by-side table - No null values shown to user - Field formatting human-readable - Differences highlighted visually

QA Sign-off: Passed

Regression Testing Tested with cars missing optional fields. Tested with extreme values (year 1900–2100, price 0–10,000,000).

Notes All combinations of fuel and transmission tested. No missing data in response.

DEF-012 — Support Ticket Chat Not Bidirectional

Fix Steps & Approach 1. Implemented `TicketResponse` entity to represent individual messages 2. Created `TicketResponseService` with add-message logic 3. Updated `SupportTicket` entity to include `List<TicketResponse> responses` 4. Implemented `POST /api/user/support-tickets/{id}/messages` endpoint 5. Added authorisation to allow ticket owner or admin to add messages 6. Created frontend message-thread UI component with chronological ordering

Testing Methodology Unit testing for message creation and retrieval. Integration testing for authorisation. Frontend component testing for thread display.

Verification Criteria - Multiple messages stored in single ticket thread - Messages appear in chronological order - Both user and admin can add messages - Non-owner user gets 403 when adding message - Ticket remains OPEN until admin closes it

QA Sign-off: Passed

Regression Testing Tested with 20+ messages per ticket. Tested rapid message posting (no race conditions). Tested both admin and user adding messages.

Notes Message timestamps accurate. No duplicate messages created.

DEF-013 — Session Token Expiration Not Enforced

Fix Steps & Approach 1. Implemented `SessionInterceptor` to validate token on each request 2. Created `SessionService` with token expiration check (120 minutes) 3. Added `SessionToken` entity with `expiresAt` timestamp 4. Implemented scheduled task to clean expired tokens every 30 minutes 5. Updated authentication to reject requests with expired tokens 6. Added error response indicating session expired for auto-logout on client

Testing Methodology Unit testing for token expiration calculation. Integration testing for interceptor validation. Load testing with many concurrent sessions.

Verification Criteria - Request with expired token returns 401 Unauthorized - Client receives 'session expired' message - Old tokens cleaned from database automatically - New login required after expiration - Session timer extends on each request

QA Sign-off: Passed

Regression Testing Tested token at 119 minutes, 120 minutes, and 121 minutes. Tested concurrent requests from expired session.

Notes No brute force vulnerability. Token cleanup runs on schedule.

DEF-014 — Car Availability Status Not Updating After Purchase

Fix Steps & Approach 1. Updated `PurchaseService` to atomically set `car.available = false` on purchase 2. Wrapped car update in transaction with order creation 3. Added database constraint to prevent

duplicate purchases 4. Implemented rollback logic if purchase fails after car update 5. Created audit trail for availability changes 6. Added test to verify car unavailable immediately after successful payment

Testing Methodology Unit testing for car availability update logic. Integration testing with concurrent purchase attempts. Database transaction testing.

Verification Criteria - Car marked unavailable immediately after successful payment - Duplicate purchase attempt returns 409 Conflict - Car remains available if payment fails - Seller cannot edit availability of purchased car - Availability status persists across sessions

QA Sign-off: Passed

Regression Testing Tested 10 concurrent purchases of same car (only 1 succeeds). Tested payment failure scenario.

Notes No race condition even with network latency. Database constraint enforced.

DEF-015 — Fraud Alert Not Triggering for High Purchase Volume

Fix Steps & Approach 1. Implemented `FraudDetectionService` with volume-based detection algorithm 2. Added `fraudAlert` flag to `PurchaseOrder` entity 3. Created check: flagged if buyer has >3 total approved purchases 4. Integrated fraud check in `PurchaseService` before order creation 5. Created admin view to see flagged orders and user purchase history 6. Implemented optional fraud notification to admin when threshold exceeded

Testing Methodology Unit testing for fraud detection logic. Integration testing with mock purchase history. Business rule testing for threshold.

Verification Criteria - Order flagged when 4th purchase by same user is made - Flag persists in database for admin review - Users with 3 purchases not flagged - Historical data considered for legacy users - Admin dashboard shows fraud alert count

QA Sign-off: Passed

Regression Testing Tested with 3 purchases (not flagged) and 4 purchases (flagged). Tested with mixed order statuses (only APPROVED counted).

Notes Configurable threshold. Performance impact minimal on order creation.